

1. Use Try-with-Resources for AutoCloseable Resources (Interview Notes)

What is Try-with-Resources?

Try-with-resources is a feature introduced in **Java 7** that **automatically closes resources** after they are used.

A resource must implement the **AutoCloseable** interface.

```
try (FileReader file = new FileReader("data.txt")) {  
    // Read file  
}
```

The `FileReader` is closed automatically, even if an exception occurs.

Why Use Try-with-Resources?

Use it whenever working with resources like:

- Files
- Database connections
- Streams
- Sockets
- Scanners

It prevents **resource leaks** and removes the need for manual cleanup.

Before Java 7

```
FileReader file = new FileReader("data.txt");  
  
try {  
    // Read file  
} finally {
```

```
file.close();  
}
```

Java 7+

```
try (FileReader file = new FileReader("data.txt")) {  
    // Read file  
}
```

No explicit finally block is needed.

Important Interview Points

- Introduced in **Java 7**.
- Automatically closes resources after the try block finishes.
- Resources must implement the **AutoCloseable** interface.
- Reduces boilerplate code and prevents resource leaks.
- Resources are closed even if an exception occurs.
- Multiple resources can be declared in the same try statement.

Example:

```
try (  
    Scanner sc = new Scanner(System.in);  
    FileReader file = new FileReader("data.txt")  
) {  
    // Use resources  
}
```

Tricky Interview Questions

Q1. What interface is required for try-with-resources?

Answer:

AutoCloseable.

Q2. When are resources closed?

Answer:

Automatically after the try block finishes, whether an exception occurs or not.

Q3. Can multiple resources be used?

Answer: Yes.

Separate them with semicolons.

Q4. Is finally required with try-with-resources?

Answer: No.

Resources are closed automatically.

Q5. Why is try-with-resources preferred over finally?

Answer:

It is cleaner, safer, and guarantees automatic resource management.

Interview Summary

- **Try-with-resources automatically manages and closes resources.**
- Works only with classes implementing **AutoCloseable**.
- Eliminates the need for manual cleanup in finally.
- Prevents resource leaks and improves code readability.
- It is the **recommended approach** for resource management in modern Java.

Interview One-Liner:

"Try-with-resources automatically closes AutoCloseable resources, making resource management safer and cleaner than using finally."

2. Use throws to Delegate Exception Handling (Interview Notes)

What is throws?

The **throws** keyword declares that a method **may throw an exception** and passes the responsibility of handling it to the **calling method**.

```
public void readFile() throws IOException {  
    FileReader file = new FileReader("data.txt");  
}
```

The method does not handle the exception itself.

Why Use throws?

Use throws when:

- The current method cannot or should not handle the exception.
 - You want the caller to decide how to handle it.
 - The exception is better handled at a higher level.
-

Example

```
public static void readFile() throws IOException {  
    FileReader file = new FileReader("data.txt");  
}  
  
public static void main(String[] args) {  
    try {  
        readFile();  
    } catch (IOException e) {  
        System.out.println("File not found.");  
    }  
}
```

Here:

- `readFile()` **delegates** exception handling.
 - `main()` **handles** it.
-

Execution Flow

Method A

↓

throws

↓

Method B (Caller)

↓

try-catch

Important Interview Points

- `throws` is used in the **method declaration**.
- It **declares** exceptions; it does **not throw** them.
- Commonly used with **checked exceptions**.
- Multiple exceptions can be declared using commas.
- The caller becomes responsible for handling or further delegating the exception.

Example:

```
public void test() throws IOException, SQLException {  
}
```

Tricky Interview Questions

Q1. Difference between throw and throws?

Answer:

- throw → Actually throws an exception object.
 - throws → Declares that a method may throw exceptions.
-

Q2. Does throws handle an exception?

Answer: No.

It only delegates responsibility.

Q3. Is throws mandatory for RuntimeExceptions?

Answer: No.

It is mainly used for **checked exceptions**.

Q4. Can one method declare multiple exceptions?

Answer: Yes.

```
throws IOException, SQLException
```

Q5. If a method declares throws, who handles the exception?

Answer:

The **calling method**, unless it also delegates using throws.

Interview Summary ★

- **throws delegates exception handling to the caller instead of handling it locally.**
- It is written in the **method signature**.
- Mainly used for **checked exceptions**.
- A method can declare multiple exceptions.
- The exception continues propagating until it is handled or reaches the JVM.

Interview One-Liner:

"throws declares that a method may throw an exception, allowing the caller to decide whether to handle or further propagate it."